Computational Intractability of Strategizing against Online Learners

author names withheld

Editor: Under Review for COLT 2025

Abstract

Online learning algorithms are widely used in strategic multi-agent settings, including repeated auctions, contract design, and pricing competitions, where agents adapt their strategies over time. A key question in such environments is how an optimizing agent can best respond to a learning agent to improve its own long-term outcomes. While prior work has developed efficient algorithms for the optimizer in special cases—such as structured auction settings or contract design—no general efficient algorithm is known.

In this paper, we establish a strong computational hardness result: unless $\mathsf{P} = \mathsf{NP}$, no polynomial-time optimizer can compute a near-optimal strategy against a learner using a standard no-regret algorithm, specifically Multiplicative Weights Update (MWU). Our result proves an $\Omega(T)$ hardness bound, significantly strengthening previous work that only showed an additive $\Theta(1)$ impossibility result. Furthermore, while the prior hardness result focused on learners using fictitious play—an algorithm that is not no-regret—we prove intractability for a widely used no-regret learning algorithm. This establishes a fundamental computational barrier to finding optimal strategies in general game-theoretic settings.

Keywords: List of keywords

1. Introduction

Online learning algorithms are often used in scenarios containing other learning agents. This includes repeated auctions where a seller and buyers learn how to construct auctions and to bid, respectively (Nekipelov et al., 2015; Noti and Syrgkanis, 2021); Repeated contracts where a contractor learns how to set up contracts and agents learn how to take actions which maximize their revenue given these contracts (Guruganesh et al., 2024); competing retailers that learn how to set prices and to adapt to the demand (Du and Xiao, 2019; Calvano et al., 2020); Information design, where a principle wants to persuade an agent to take actions by deciding which information to share (Chen and Lin, 2023); and many more examples exist. In all these scenarios, the agents are operating in the same environment and influence each other's utilities.

In this paper we assume that there are two agents, a *learner* and an *optimizer*, and we ask:

Given that the learner is using a well-known learning algorithm to decide on his course of action, what is the best strategy for the optimizer to maximize her own reward (a.k.a. utility)?

We formalize this setting under the lens of *game theory*: we assume that there exists some *finite normal-form game* between a learner and an optimzer, as explained below. Concretely, this means that the optimizer can take actions from some set finite $\mathcal{A}$, the learner can take actions from some finite set $\mathcal{B}$, and there are functions A and B which determine the reward of the agents: if the optimizer plays action $\mathbf{x} \in \mathcal{A}$ and the learner plays action $\mathbf{y} \in \mathcal{B}$, the optimizer receives the reward $A(\mathbf{x}, \mathbf{y}) \in [-1, 1]$ and the learner receives the reward $B(\mathbf{x}, \mathbf{y}) \in [-1, 1]$. We assume *repeated game playing*: this means that in every iteration $t = 1, \ldots, T$, the optimizer chooses an action $\mathbf{x}(t)$,

the learner chooses an action $\mathbf{y}(t)$, and the agents receives reward $A(\mathbf{x}(t), \mathbf{y}(t))$ and $B(\mathbf{x}(t), \mathbf{y}(t))$, respectively. The goal of each agent is to maximize their cumulative reward, summed over all the T iterations. For this purpose, the learner is using a standard online learning algorithm which determines which action to take in each iteration, whereas the optimizer's goal is to play according to a strategy that maximizes her cumulative reward given the learner's algorithm.

Perhaps the most fundamental notion by which online learning algorithms are measured is *regret*. That is, the difference between the cumulative reward of the learner and the reward that they would gain if they played a fixed action and if the optimizer played the same sequence of actions, which is defined as

$$\sum_{t=1}^{T} B(\mathbf{x}(t), \mathbf{y}(t)) - \max_{\mathbf{y} \in \mathcal{B}} \sum_{t=1}^{T} B(\mathbf{x}(t), \mathbf{y}) \ .$$

A learning algorithm is no-regret if its regret behaves as $o(T)$ and no-regret is a desirable property. A comprehensive theory has been developed around no-regret learning algorithms, and this includes the fundamental Hedge algorithm, also known as Multiplicative Weights Update (MWU), which is a simple algorithm that attains the asymptotically-optimal regret (Littlestone and Warmuth, 1994; Freund and Schapire, 1997; Arora et al., 2012).

A disadvantage of the notion of regret is that it does not take into account scenarios where the other agent (optimizer, in our case) utilizes an adaptive algorithm. In spite of that, standard no-regret learners are still commonly used. This raises the question of how an optimizing agent can behave in environments where other agents use no-regret learning algorithms to make decisions, to maximize its reward. Braverman et al. (2018) showed that in the context of repeated auctions between one seller and one buyer, if the buyer is using a mean-based no-regret learning algorithm (such as Multiplicative Weights Update) then the seller can extract full welfare, which is more than what they could get if the learner was strategic. This line of work has been generalized in scenarios with one (Deng et al., 2019a; Rubinstein and Zhao, 2024) or multiple no-regret buyers (Cai et al., 2023). Beyond mechanism design, similar interactions between a learner and an optimizer were studied in repeated contracts (Guruganesh et al., 2024) and in repeated information design (Bayesian Persuation) (Lin and Chen, 2024). In all these scenarios, an asymptotically-optimal strategy for the optimizer was developed. However, the solution in each instance was limited to the special game structure at hand. In general games, Deng et al. (2019b) has shown that the general optimization problem is essentially equivalent to solving a high-dimensional optimal control problem, however they do not provide an efficient algorithm. In this paper we show, in fact, that this is no coincidence: there is no efficient algorithm for the optimizer in general games, thereby resolving an open question of Guruganesh et al. (2024).

1.1. Our Contribution

Unlike prior work that we discussed above that has provided efficient algorithms only in structured settings or restricted cases, we establish a general computational hardness result. We prove that unless $\mathsf{P} = \mathsf{NP}$, no polynomial-time optimizer can compute a near-optimal strategy against a learner that uses a standard no-regret algorithm, specifically the Hedge/MWU. This formally establishes that no general efficient strategy exists for the optimizer in repeated game settings.

We formalize the optimization problem that we study. We assume that the learner's algorithm is Hedge (MWU), and it is parameterized by a step-size $\eta \geqslant 0$ (cf. Definition 2). Any sequence of

optimizer's actions $\mathbf{x}(1), \ldots, \mathbf{x}(T)$ together with the learning rate η determine the learner's actions. This deremines the optimizer's reward: $R(\{\mathbf{x}(t) : t \geqslant 1\}) = \sum_{t=1}^{T} A(\mathbf{x}(t), \mathbf{y}(t))$. The optimizer's goal is:

Task 1 *Given $T \in \mathbb{N}$, given truth tables of the reward functions $A, B \colon \mathcal{A} \times \mathcal{B} \to [-1, 1]$ and given a step-size η for the learner, find a sequence of actions of the optimizer $x(1), \ldots, x(T) \in \mathcal{B}$ that approximately maximizes its reward $R\{\mathbf{x}(t) : t \geqslant 1\}$.*

Theorem 1 *Fix absolute constants $\alpha, \beta \in (0, 1]$. Let T denote the horizon, suppose the learner plays Hedge with learning rate parameterized as $\eta = 1/T^{1-\alpha}$ (cf. Definition 2) and suppose that the number of strategies per player is bounded by $|\mathcal{A}|, |\mathcal{B}| \leqslant T^{\beta}$. If $\mathsf{P} \neq \mathsf{NP}$, there exists no algorithm whose runtime is polynomial in T for Task 1 that outputs $\mathbf{x}(1), \ldots, \mathbf{x}(T)$ such that:*

$$R(\{x(t) : t \geqslant 1\}) \geqslant \max_{\{x^\star(t) : t \in [T]\}} R(\{x^\star(t) : t \in [T]\}) - cT, \tag{1}$$

for an absolute constant $c > 0$.

Our result significantly strengthens the hardness result of Assos et al. (2024), who show a computational hardness result for attaining the optimal reward in general two-player games. However their result had two significant limitations: (1) their impossibility result is based on approximating the reward of the optimizer up to an additive $\Theta(1)$-factor, whereas ours hold even for approximation up to $\Omega(T)$; (2) Their result assumed that the learner uses *follow-the-leader* (a.k.a. fictitious play), an algorithm that is not no-regret.

There are other differences which are not merely conceptual: the structure of the game considered in Assos et al. (2024) has learner action space of size T. Therefore establishing any (even $\Omega(T)$) hardness for such instances, does not preclude, for instance, the existence of an efficient algorithm which collects cumulative reward within $O(\sqrt{|\mathcal{B}|T})$ of that of the best optimizer. Further, it relied on the instability of the learner, which can change actions in each iteration, whereas we consider the MWU that is slow-changing, as is the case of most no-regret learners.

In contrast, our results show that even when the size of the game is an arbitrarily small (but constant) power of T, the regret scaling is $\Omega(T)$, thus ruling out the existence of such algorithms. In a sense, this is the best achievable: if the action space were to be any smaller, say polylogarithmic, the optimizer is now allowed to carry out superpolynomial compute in the size of the game. This essentially rules out approaches which prove hardness based on reducing to NP-complete problems of size poly($|\mathcal{B}|$), under $\mathsf{P} \neq \mathsf{NP}$. It is conceivable that under stronger assumptions, like ETH, using the same techniques, hardness results may be established even when the size of the game is yet smaller than even a polynomial in T.

From a technical point of view, our hardness reduction appeals to a special case of the traveling salesman problem with integral weights, $(1, 2)$-TSP, for which constant factor hardness of approximation is known Karpinski and Schmied (2012). Our result shows $\Omega(T)$ hardness of optimizing against MWU, even when the learning rate is as small as $1/T^{0.99}$, which is the regime where the learner is very slow changing[1]. When the learning rate is 0, the problem becomes easy, since the learner is no longer adaptive.

1. 0.99 can be made any constant arbitrarily close to 1

1.2. Related work

Multi-Agent Learning and Strategic Optimization The study of learning in multi-agent environments has a long history in game theory (Cesa-Bianchi and Lugosi, 2006; Youn, 2004; Freund and Gittins, 1998). A central question in this field is whether repeated interactions between learning agents lead to convergence to equilibrium (Robinson, 1951; Cesa-Bianchi and Lugosi, 2006). Classical results establish that when agents repeatedly play against each other using learning algorithms, their average play can converge to various equilibrium concepts. More recent work has shown that when all agents use similar learning algorithms, play can converge significantly faster to equilibria compared to classical results (Syrgkanis et al., 2015; Daskalakis et al., 2021; Anagnostides et al., 2022a,b; Farina et al., 2022; Piliouras et al., 2022; Zhang et al., 2023a). However, a different challenge arises when a strategic optimizer interacts with a learning agent: what is the best strategy for the optimizer to maximize long-term reward?

Optimizing Against Learning Agents in Structured Settings A series of works have studied how an optimizer should act when interacting with a learning agent in structured settings. In auction design, it has been shown that a seller interacting with a no-regret buyer—whether a single buyer or multiple buyers—can extract nearly the entire welfare, achieving an additional $\Omega(T)$ reward compared to what would be possible against a fully strategic buyer (Braverman et al., 2018; Deng et al., 2019a; Cai et al., 2023; Rubinstein and Zhao, 2024). A similar setting was studied in contract design, where a contractor optimally sets contracts for an agent using a learning algorithm (Guruganesh et al., 2024), and in Bayesian persuasion, where an information designer persuades a learning agent by strategically revealing information (Lin and Chen, 2024). In each of these cases, efficient algorithms were developed for the optimizer, leveraging the structure of the specific problem.

General Games: Lack of Efficient Optimizer Algorithms Beyond structured problems, several works have studied optimization against learning agents in general games, where the optimizer's goal is to maximize its reward in an arbitrary repeated game. Deng et al. (2019b) showed how to find an approximately optimal strategy for the optimizer. However, their work only provided a reduction to an optimal control problem and did not provide an efficient algorithm to solve it. Other works have explored similar settings in Bayesian games (Mansour et al., 2022), Pareto-optimal interactions (Arunachaleswaran et al., 2024), and optimizers facing arbitrary no-regret learners (Brown et al., 2024), but none of these provide a general efficient algorithm for the optimizer.

Computational Hardness in Special Cases While no efficient algorithm is known in general settings, some restricted cases have been solved efficiently. Guo and Mu (2023) developed a polynomial-time algorithm for an optimizer interacting with an MWU learner in two-player games where each agent has only two actions. Masoumian and Wright (2024) provided an efficient algorithm for an optimizer interacting with a learner that has small bounded memory. Collina et al. (2023) showed that if a learner best-responds to a known optimizer policy, then the optimizer can achieve more than the per-round Stackelberg value, and they provided efficient algorithms to do so. In first-price auctions, Kumar et al. (2024) constructed a bidding algorithm that is both no-regret and robust against a strategic seller.

Learning the Stackelberg Equilibrium from Repeated Interactions The Stackelberg equilibrium represents the best possible reward an optimizer can obtain in a one-shot game, assuming the learner best-responds to the optimizer's actions. The problem of computing the Stackelberg equilibrium has been widely studied (Conitzer and Sandholm, 2006; Peng et al., 2019; Collina et al., 2023). In repeated interactions, prior work has studied whether an optimizer can learn the Stackelberg equilibrium when it does not know the learner's reward function in advance. Ananthakrishnan et al. (2024) showed an impossibility result when the learner is strategic and the optimizer has missing information. Other works have developed algorithms for learning a Stackelberg strategy in repeated games under similar uncertainty (Balcan et al., 2015; Marecki et al., 2012; Lauffer et al., 2022; Haghtalab et al., 2022; Brown et al., 2024). Additionally, Zhang et al. (2023b) explored how a principal can influence online learners to converge to specific equilibria.

2. Preliminaries

Repeated Games Notation. Consider a two-player game, where the optimizer's and learner's strategy sets are $\mathcal{A}$ and $\mathcal{B}$, respectively. The players are allowed to play distributions over their action sets (a.k.a. mixed strategies). At time t the optimizer and the learner play $\mathbf{x}(t) \in \Delta(\mathcal{A})$ and $\mathbf{y}(t) \in \Delta(\mathcal{B})$, respectively. The reward functions for the learner and the optimizer are parameterized by matrices $\boldsymbol{A}, \boldsymbol{B} \in [-1,1]^{|\mathcal{A}|\times|\mathcal{B}|}$. The reward collected by the optimizer and the learner at time t is $\mathbf{x}(t)^\top \boldsymbol{A}\mathbf{y}(t)$ and $\mathbf{x}(t)^\top \boldsymbol{B}\mathbf{y}(t)$, respectively. The notation $\mathbf{x}(a,t)$ denotes the probability of action $a \in \mathcal{A}$ at time t under $\mathbf{x}(t)$. Likewise, the notation $\mathbf{y}(b,t)$ is defined. The repeated game is assumed to be played over a horizon of length T. Furthermore, for $t \in [T]$, let,

$$\forall b \in \mathcal{B}, \quad r_{\text{learner}}(b,t) = \sum_{t'=1}^{t} \mathbf{x}(t')^\top \boldsymbol{B}\delta_b \tag{2}$$

denote the cumulative rewards of the actions of the learner at time t.

Definition 2 (Hedge/MWU learner) *Fix the step size $\eta \geqslant 0$ of the algorithm. In a repeated game with matrices $\boldsymbol{A}, \boldsymbol{B}$ for optimizer and learner, and an initial reward history $r_0 \in \mathbb{R}^m$, plays a mixed strategy, where at time t, for any $b \in \mathcal{B}$,*

$$\boldsymbol{y}(b,t) = \frac{\exp\big(\eta(r_{learner}(b,t) + r_0(b))\big)}{\sum_{b' \in \mathcal{B}} \exp\big(\eta(r_{learner}(b',t) + r_0(b'))\big)} \tag{3}$$

The parameter η, is referred to as the learning rate.

With notation in place, we define the main problem we study in this paper.

Definition 3 (Optimizing against MWU) *For a sequence of pure optimizer strategies $\{\boldsymbol{x}(t) : t \in [T]\}$, we will denote $R_{\eta,r_0}(\{\boldsymbol{x}(t) : t \geqslant 1\})$ as the cumulative reward collected by the optimizer, when playing against multiplicative weights with learning rate and initialization r_0. The problem of optimizing against MWU is defined as: find the sequence of pure strategies $\{\boldsymbol{x}(t) : t \in [T]\}$ which maximizes $R_{\eta,r_0}(\{\boldsymbol{x}(t) : t \geqslant 1\})$.*

We notice that the standard definition of MWU is with $r_0 = 0$ and indeed we will prove hardness for this case. To show this, we will first prove a hardness when $r_0 > 0$ and provide a reduction between these two scenarios.

2.1. Reduction Workhorse: $(1,2)$-TSP

To prove the hardness result (Theorem 1) we will reduce the problem of optimizing against MWU, to an NP hard problem with an approximation hardness guarantee: we consider a variant of the traveling salesman problem, where the edge weights of the graph are restricted to be either 1 or 2.

Definition 4 ($(1,2)$-maxTSP) *An instance of $(1,2)$-TSP specified by $G = (V, W)$ is in one-one correspondence with an instance of $(1,2)$-maxTSP on $\widetilde{G} = (V, \widetilde{W})$ by the relation $\widetilde{W}(e) = 3 - W(e)$. This corresponds to replacing all the weight-2 edges by weight-1 edges, and vice versa. The objective is to find an Hamiltonian cycle in $\widetilde{G}$ with maximum weight.*

The following theorem shows constant approximation factor hardness for $(1,2)$-maxTSP and is proved in the Appendix (this follows trivially from Karpinski and Schmied (2012)).

Theorem 5 *Unless $\mathsf{P} = \mathsf{NP}$, there is no polynomial time approximation algorithm for $(1,2)$-maxTSP achieving a multiplicative approximation factor of $1067/1068 + \epsilon$ for any $\epsilon > 0$.*

3. Lower bound against MWU with non-zero reward history

The main result we show in this section is an $\Omega(T)$ hardness result for optimizing against MWU when initialized with an arbitrary reward history. In the subsequent section, we will extend this result to optimizing against MWU in the absence of any initialization.

3.1. Structure of the reduction

Define $E = (V \times V) \setminus \{(v,v) : v \in V\}$, the set of directed edges of the complete digraph on V. Given an instance of $(1,2)$-maxTSP on the vertex set V, specified by a set of edge weights $\{W_e : e \in E\}$, define the following game structure for both agents:

$$\textbf{Optimizer's action space } \mathcal{A}_{\text{init}} = E \textbf{ of size } m_{\text{init}} = |V|^2 - |V| \tag{4a}$$

$$\textbf{Learner's action space } \mathcal{B}_{\text{init}} = E \cup V_{\text{in}} \cup V_{\text{out}} \textbf{ of size } n_{\text{init}} = |V|^2 + |V| \tag{4b}$$

where V_{in} and V_{out} are auxiliary copies of V. For any vertex $v \in V$, we will use v_{in} and v_{out} to denote the corresponding vertex in V_{in} and V_{out}, and vice versa. Recall that we parameterize MWU's learning rate as $\eta = 1/T^{1-\alpha}$ for some $\alpha \in (0,1]$. We will later choose V to be of size $n = T^{\min\{\alpha,\beta\}/3}$. Thus, the learner's and optimizer's action spaces are of size $O(T^{2\beta/3})$.

The reduction's idea is to create a game instance, where the optimal strategy for the optimizer is to play along some approximate solution of the maxTSP problem. Specifically, some edge e^+ is forced as the first edge to play, and the optimizer has to proceed with an approximate solution to the TSP problem, playing each edge along this path for k rounds. The reward matrix will be constructed such that, when the optimizer plays along such strategy, it receives a utility that is proportional to its path weight, hence it would like to find a maximal-weight path. Notice that the optimizer does not have to play according to some path. To ensure that it cannot gain more utility deviating from playing each edge in a path for k rounds, the historical rewards are defined such that if the learner significantly diverges from such a play-structure, the learner will play actions in V_{in} and V_{out} from that point onwards, and these yield no utility for the optimizer.

The game matrices are defined as follows:

$$\text{For } (w,x) \in E, \quad \boldsymbol{A}(\underbrace{(u,v)}_{\text{opt.}}, \underbrace{(w,x)}_{\text{lr.}}) = \begin{cases} W_{(w,x)} & \text{if } (u,v) = (w,x) & \text{(5a)} \\ W_{(w,x)} & \text{if } u = x, & \text{(5b)} \\ 0 & \text{otherwise.} & \text{(5c)} \end{cases}$$

$$\text{For } a \in \mathcal{A}_{\text{init}},\ b \in \mathcal{B}_{\text{init}} \setminus E, \quad \boldsymbol{A}(a,b) = 0. \tag{6}$$

Thus, if the learner plays an edge e, the optimizer collects W_e units of reward for either responding with the same edge or responding with one which stems from the out-vertex of this edge. The optimizer surely collects no reward if the learner had chosen an action in V_{in} or V_{out}. On the other hand, for some $\varepsilon > 0$ and an arbitrary edge $e^\dagger = (u^\dagger, v^\dagger) \in E$, the game matrix for the learner is,

$$\text{For } (w,x) \in E, \quad \boldsymbol{B}(\underbrace{(u,v)}_{\text{opt.}}, \underbrace{(w,x)}_{\text{lr.}}) = \begin{cases} 0 & \text{if } (u,v) = e^\dagger & \text{(7a)} \\ 1 & \text{if } (u,v) \neq e^\dagger \text{ and } (w,x) = (u,v) & \text{(7b)} \\ -\varepsilon & \text{if } x = u, & \text{(7c)} \\ 0 & \text{otherwise.} & \text{(7d)} \end{cases}$$

$$\text{For } b \in V_{\text{in}}, \quad \boldsymbol{B}((u,v),b) = 2\mathbb{I}(b = v) \tag{8}$$

$$\text{For } b \in V_{\text{out}}, \quad \boldsymbol{B}((u,v),b) = 2\mathbb{I}(b = u) \tag{9}$$

Thus, the cumulative reward of an edge for the learner is the number of times the same edge was played by the optimizer minus the number of times the optimizer chooses an edge emanating from its out vertex scaled by ε.

In this section, we will prove the result assuming that the cumulative rewards of actions played by MWU are initially non-zero. This "initial reward" influences the probability of actions chosen by MWU. In particular, for $k > 0$ to be determined later, suppose the cumulative rewards are initialized as follows,

$$\text{For } v_{\text{in}} \in V_{\text{in}},\ r_0(v_{\text{in}}) = -k \tag{10}$$

$$\text{For } v_{\text{out}} \in V_{\text{out}},\ r_0(v_{\text{out}}) = -k \tag{11}$$

$$\text{For } (u,v) \in E \setminus \{e^\dagger\},\ r_0((u,v)) = 0 \tag{12}$$

$$r_0(e^\dagger) = k \tag{13}$$

In particular, when $k \gg \log(n)/\eta$, the initial distribution determined by MWU places most of its mass on the edge $e^\dagger$. As we will see later, the presence of the edge $e^\dagger$ is vital: it will ensure that there always exists an action with high cumulative reward ($\approx k$) for the learner. On the other hand, the vertices $v_{\text{in}} \in V_{\text{in}}$ and $v_{\text{out}} \in V_{\text{out}}$ will serve to keep a handle on the number of edges the optimizer may play sinking / emanating out of a single vertex. These vertices begin with a high negative reward, $-k$, but gain reward at twice the rate of edges in $\mathcal{B}_{\text{init}}$. Once any of these vertices gain too much reward, MWU will displace most of its mass toward such actions, preventing the optimizer from gaining significant reward then onward.

3.2. Proof of Theorem 1 when learner has non-zero historical rewards

We consider some horizon length $T_{\text{init}} \leqslant T$. We will also implement the proof so that the only constraint on the value of k is that it falls into some range depending on other fixed problem parameters. When reducing the case without initialization to the case with initialization, the choice of T_{init} will be made by the optimizer, although it must essentially be $\approx \varepsilon T$ for it to be able to collect the most reward.

We start by arguing that the optimizer should not play an edge significantly more than k times, otherwise she will stop gaining reward. Define $V_{\text{excess}}(\Delta, t)$ as the set of vertices v for which there exists an incident edge that has been played more than $k + \Delta$ times by the optimizer by time t, where Δ is small compared to k.

Definition 6 *For $v \in V$ and $t = 1, \ldots, T$ denote by $d_{in}(v, t)$ (resp. $d_{out}(v, t)$) the number of times that the optimizer played an edge incoming (resp. outgoing) v, in iterations $1, \ldots, t$. We refer to $d_{in}(v, t)$ and $d_{out}(v, t)$ as in-degree and out-degree of v throughout the play. Let $V_{excess}(\Delta, t)$ denote the set of vertices which have in-degree or out-degree exceeding $k + \Delta$. Namely,*

$$V_{excess}(\Delta, t) = \{v \in V : \max\{d_{in}(v, t), d_{out}(v, t)\} \geqslant k + \Delta\}\,. \tag{14}$$

In the following lemma we prove that the optimizer will be punished for playing edges that emanate or sink in a vertex v too many times. Specifically, if at time $t_{\max}$, $V_{\text{excess}}(\Delta, t)$ is non empty, then from that point onwards the rewards the optimizer can obtain are limited. The idea is that if the optimizer exceeds the threshold of $k + \Delta$ for a vertex v, one of the special actions $v_{\text{in}}, v_{\text{out}}$ will receive much higher reward than the other actions of the optimizer. The MWU learner will then place almost all of its probability mass on this action, for which the optimizer earns no reward.

Corollary 7 *Suppose that the first time $V_{excess}(\Delta, t)$ becomes non-empty is at time $t_{\max}$, and so far the optimizer has collected reward $R(t_{\max})$. Then, by the end of the repeated interaction the optimizer can earn at most $R(t_{\max}) + 1$ rewards (for appropriate value of Δ).*

Next, we argue that the optimizer will not gain much reward playing edges emnating non-heavy vertices — where a non-heavy vertex is one that the optimizer played at least $k - \Delta$ times some edge incoming to it. Define $S_{\text{heavy}}(\Delta, t)$, the set of *heavy vertices* at time t. This is the union of $\{v^\dagger\}$ with the vertices $v \in V$ which satisfy the condition that there exists an edge of the form $e = (\cdot, v)$ such that $E(e, t) \geqslant k(1 - \varepsilon) - \Delta$. Since we restrict ourselves to time $t_{\max}$, vertices have in-degree at most $k + \Delta$ (as induced by the edges played by the optimizer). Thus, the heavy vertices are those with high in-degree, induced almost entirely by $\approx k$ copies of a single incoming edge.

Definition 8 *Define $S_{\text{heavy}}(\Delta, t)$ to be the set of vertices $v \in V$ for which there exists an edge $(\cdot, v)$ for which this edge has been used by the optimizer at least $k(1 - \varepsilon) - \Delta$ times plus $v^\dagger$. Precisely:*

$$S_{\text{heavy}}(\Delta, t) = \{v^\dagger\} \cup \{v \in V | \exists u \in V, Q((u, v), t) \geqslant k(1 - \varepsilon) - \Delta\} \tag{15}$$

where $Q((u, v), t)$ is the number of time that (u, v) was played by the optimizer in iterations $1, \ldots, t$.

In Lemma 9, we argue that if we consider the set of heavy vertices, $S_{\text{heavy}}(\Delta, t_{\max})$, the optimizer essentially only collects reward for the edges played which emanate from a vertex in this set. We will denote this cumulative reward by $r_{\text{heavy}}(\Delta, t_{\max})$. The idea is that any vertex $v \notin S_{\text{heavy}}(\Delta, t_{\max})$ will never have a single edge played more than $k(1 - \varepsilon) - \Delta$ times incoming into it. The optimizer may only collect reward for an edge emanating from this vertex if:

1. MWU places high mass on some edge incoming into this vertex. This is not possible because every edge incoming into v has low frequency (as induced by the optimizer).
2. MWU places high mass on the same edge. This is not possible until the same edge has been pulled many (at least approximately $k(1-\varepsilon)$) times by the optimizer. After this point, the same edge can be played at most until its frequency hits $k+\Delta$. In the small interim interval is when any reward can be collected for picking this edge.

Lemma 9 *Let $r_{\text{heavy}}(\Delta, t)$ count the contribution of the total reward collected by the optimizer at time t arising only from edges which emanate from a vertex in $S_{\text{heavy}}(t_{\max})$. Then,*

$$r_{optimizer}(t_{\max}) \leqslant r_{\text{heavy}}(\Delta, t_{\max}) + 2n(k\varepsilon + 2\Delta) + 3 \tag{16}$$

The above assertions have a few consequences. First, we have that the optimizer should be very careful not to exceed playing one action more than $k+\Delta$ times, and risks receiving almost no reward for the rest of the game (Corollary 7). We then define the 'heavy' vertices to be the vertices for which an edge incident to them have been played a significant amount of times. Then, with Lemma 9, we argued that it is enough to look at the reward of the edges emanating from 'heavy' vertices of the optimizer, $r_{\text{heavy}}(\Delta, t_{\max})$, as they make up most of the reward of the optimizer.

We will now try to unpack and upper bound $r_{\text{heavy}}(\Delta, t_{\max})$. Notice that for every $v \in S_{\text{heavy}}(\Delta, t_{\max})$, there is a unique vertex u for which the action (u, v) is played more than $k(1-\varepsilon)-\Delta$ times. There cannot be more than one, otherwise we would have $d_{\text{in}}(v, t) = 2(k(1-\varepsilon)-\Delta) > k+\Delta$, which is a contradiction to the fact that V_{excess} is empty prior to reaching time $t_{\max}$. Similarly, one can prove that for every vertex in v there can be at most one action (v, w) having reward at least $k(1-\varepsilon)-\Delta$. This motivates the following graph construction of G'.

Definition 10 *Construct a graph G' using the vertices of V as follows; add an edge from $u \to v$ if $v \in S_{\text{heavy}}(\Delta, t_{\max})$ and $E((u, v), t_{\max}) \geqslant k(1-\varepsilon)-\Delta$.*

The first observation, as already hinted, is that G' is a graph in which all vertices have in-degree and out-degree at most one. That means that we may split G' into disjoint paths and cycles, which we denote as $C_1, C_2, \cdots C_s$. Our goal will now be to bound the rewards obtained by actions emanating from the paths (Lemma 11) and cycles (Lemma 12) of the Graph G' and combining the two to get a global upper bound of the reward of the optimizer (Lemma 13), emanating from vertices in $S_{\text{heavy}}(\Delta, t)$. We begin by bounding the rewards of actions/edges that emanate from vertices of a specific path of the graph G'; We argue that the total reward collected by the optimizer for edges emanating from these vertices is approximately upper bounded by the sum of the weights (in the maxTSP instance) of the edges along this path along with a free edge pointing from the last vertex to an arbitrary vertex, multiplied by k. Without loss of generality, we assume that the component C_1 contains the edge $e^{\dagger}$.

Lemma 11 *Consider any path C_i in G' composed of vertices $z_1, z_2, \cdots, z_m \in V$. The total reward collected by the optimizer for edges which stem from vertices on C_i is upper bounded by,*

$$\sum_{i=1}^{m} (k(1+\varepsilon) + 3\Delta) W(z_i, z_{i+1}) + 2(k+\Delta)\mathbb{I}(i=1). \tag{17}$$

where z_{i+1} is an arbitrary (free) vertex.

We continue by bounding the rewards of actions/edges that emanate from vertices of a specific cycle of the graph G'; we show that the total reward collected by the optimizer for edges emerging from vertices in C_i is approximately upper bounded by the weight of edges in some $|C_i| - 1$ length path within C_i up to a factor of k.

Lemma 12 *Consider any cycle C_i in G' composed of vertices $z_1, z_2, \cdots, z_m$. The total reward collected by the optimizer for edges which stem from vertices on C_i is upper bounded by,*

$$(k(1+3\varepsilon)+7\Delta)\sum_{i\in[m]\setminus\{i^\star\}} W(z_i, z_{i+1}) + 2(k+\Delta)\mathbb{I}(i=1) + \frac{1}{n} \tag{18}$$

for some $i^\star \in [m]$, where $z_{m+1} \triangleq z_1$.

We now combine the above to get an upper bound on the total reward heavy edges can contribute. We argue that we can combine the edges of graph G' to form a Hamiltonian Cycle without losing any reward. Using this fact, we can then upper bound the contribution of edges emanating from the heavy vertices with the reward of the Hamiltonian Cycle we obtain when combining all the heavy vertices. This, together with Corollary 7 and Lemma 9 gives the following result.

Lemma 13 *Assume $\Delta \geqslant \frac{1}{\eta}\log\left(2n^2 T_{init}\right)$. Then, the reward collected by the optimizer at the end of the repeated interaction is at most,*

$$(k(1+5\varepsilon)+9\Delta)\sum_{i=1}^{n} W(z_i, z_{i+1}) + (2k+2\Delta+4) \tag{19}$$

where $z_1 \to z_2 \to \cdots \to z_n \to z_1$ is some Hamiltonian cycle on the vertex set V, and $z_{n+1} = z_1$. This upper bound applies for any value of horizon T_{init}, as long as Δ is suitably large.

The last step is to show that there exists an optimizer strategy which collects reward approximately lower bounded by the weight of the maximum weight Hamiltonian cycle (i.e., the solution to the maxTSP instance) multiplied by k. This is established in Lemma 14.

Lemma 14 (Bounding the reward of the best optimizer) *Consider any maximum weight Hamiltonian cycle in G, defined by the sequence of vertices $z_1^\star \to z_2^\star \to \cdots \to z_n^\star \to z_1^\star$. As long as $T_{init} \geqslant nk$, there exists an optimizer strategy such that the total reward collected is at least,*

$$(1-\varepsilon)^2 k \times \left(1 - \frac{2}{n}\right)\sum_{i=1}^{n} W(z_i^\star, z_{i+1}^\star), \tag{20}$$

where $z_{n+1}^\star = z_1^\star$, and assuming that $\eta\varepsilon k \geqslant \frac{1}{\varepsilon-\varepsilon^2}\log\left(n^3\right)$.

We now have all we need to complete the proof.

Proof sketch of the theorem. Suppose we could approximate the optimal rewards of the optimizer; by Lemma 13 we know that the optimizer can achieve rewards up to $k(1+\epsilon_1)$ times the length of a Hamiltonian cycle in the maxTSP instance, where ϵ_1 can get arbitrarily close to 0. On the other hand, we know by Lemma 14 that the best the optimizer can do is at least $k(1-\epsilon_2)$ times the length of the heaviest Hamiltonian cycle in the maxTSP instance, where ϵ_2 can also get arbitrarily close to 0. By Theorem 5, we know that unless $\mathsf{P} = \mathsf{NP}$, it is impossible to approximate the weight of the optimal Hamiltonian cycle to within a $(1+c)$-factor for some $c > 0$. This implies that the rewards of the optimizer cannot be to close to optimal, and thereby leads to the $\Omega(T_{\text{init}})$ hardness guarantee.

4. Lower bound against MWU: removing the initialization

In this section, we will show how to reduce the case of MWU without initialization to the case with initialization. To do this, we will modify the structure of the game discussed in Section 3.1 for optimizing against MWU with the initialization of the form eqs. (10) to (13) and add a few additional actions. For $p \in \mathbb{N}$ to be determined later,

$$\textbf{Optimizer's action space } \mathcal{A} = \mathcal{A}_{\text{init}} \cup \{\blacklozenge\} \textbf{ of size } m = |V|^2 - |V| + 1 \tag{21a}$$

$$\textbf{Learner's action space } \mathcal{B} = \mathcal{B}_{\text{init}} \cup \widetilde{\mathcal{B}}_{\text{init}} \textbf{ of size } n = (p+1)(|V|^2 + |V|) \tag{21b}$$

The optimizer's action space is augmented by a single special action $\blacklozenge$. On the other hand, the learner's action space is augmented by $\widetilde{\mathcal{B}}_{\text{init}}$, which contains p copies of each action in $\mathcal{B}_{\text{init}}$. Each of these p copies will behave symmetrically with respect to the game matrices $\boldsymbol{A}$ and $\boldsymbol{B}$, and so we will simply refer to the "counterpart" of any $b \in \mathcal{B}_{\text{init}}$ as any one of its copies in $\widetilde{\mathcal{B}}_{\text{init}}$. We will choose $p = T^{\beta/3}$, which will bring the size of the learner's action space to be $O(T^{\beta})$.

The game matrices are amended as follows. For any learner edge $\widetilde{b} \in \widetilde{\mathcal{B}}_{\text{init}}$ and any action $a \in \mathcal{A}_{\text{init}}$,

$$\boldsymbol{A}(a, \widetilde{b}) = 0. \tag{22}$$

Furthermore, the optimizer collects no reward for playing $\blacklozenge$. Namely,

$$\text{For all } b \in \mathcal{B},\ \boldsymbol{A}(\blacklozenge, b) = 0. \tag{23}$$

From the point of view of the learner, the actions in $\mathcal{B}_{\text{init}}$ and $\widetilde{\mathcal{B}}_{\text{init}}$ are not symmetric. For a small constant $\beta > 0$ to be determined later, and any $b \in \mathcal{B}_{\text{init}}$,

$$\boldsymbol{B}(\blacklozenge, b) = \begin{cases} k^\star/T & \text{if } b = e^\dagger \\ -k^\star/T & \text{if } b \in V_{\text{in}} \cup V_{\text{out}} \\ 0 & \text{otherwise.} \end{cases}, \tag{24}$$

Where $k^\star = \varepsilon(1-\varepsilon)^{-1}T^{1-\min\{\alpha,\beta\}/3}$. For any $b \in \mathcal{B}_{\text{init}}$ and its counterpart $\widetilde{b} \in \widetilde{\mathcal{B}}_{\text{init}}$,

$$\boldsymbol{B}(\blacklozenge, \widetilde{b}) = \boldsymbol{B}(\blacklozenge, b) - \gamma, \tag{25}$$

where $\gamma = \beta/3(1-\varepsilon)^{-1}T^{-\alpha}\log(T)$. On the other hand, for any learner action $b \in \mathcal{B}_{\text{init}}$ and its set of counterparts $\widetilde{b} \in \widetilde{\mathcal{B}}_{\text{init}}$, and any optimizer edge $a \in \mathcal{A}_{\text{init}}$,

$$\boldsymbol{B}(a, \widetilde{b}) = \boldsymbol{B}(a, b) \tag{26}$$

4.1. Proof sketch of Theorem 1

The intuition behind the amended game structure is as follows. In the initial iteration, observe that the presence of $p \gg 1$ copies of each action in $\mathcal{B}_{\text{init}}$ ensures MWU does not place a significant amount of probability mass on actions in $\mathcal{B}_{\text{init}}$. This in turn ensures that the optimizer cannot collect much reward in the first step. In order to mitigate this issue in future steps, notice that whenever the optimizer plays $\blacklozenge$, the reward on all actions in $\widetilde{\mathcal{B}}_{\text{init}}$ decreases a little bit. Once $\blacklozenge$ is played sufficiently many times, the actions in $\widetilde{\mathcal{B}}_{\text{init}}$ are downweighted enough to the point where

MWU no longer places a significant amount of mass on them; the optimizer can begin to collect reward after this point. How many times should the optimizer play ♦ before this starts to happen?

It will turn out that the answer to this question will be $\approx (\eta\beta)^{-1}\log(p)$ times. Furthermore, the choice of γ is reverse engineered from the equation $(\eta\beta)^{-1}\log(p) = (1-\varepsilon)^2 T$. Thus, a good optimizer would need to play the action ♦ $\approx (1-\varepsilon)^2 T$ times before the effect of the actions in $\widetilde{\mathcal{B}}_{\text{init}}$ is nullified. This leaves out a small $\approx 2\varepsilon T$ portion of the horizon where the optimizer may collect rewards. By virtue of the structure of the game, however, the action $e^{\dagger}$ has collected $\approx k^\star$ cumulative reward by this point, and likewise, actions in $V_{\text{in}} \cup V_{\text{out}}$ have cumulative reward $\approx -k^\star$. Observe the resemblance to the initialization in eqs. (10) to (13): we may now use techniques for bounding the reward of the optimizer for the case of MWU with initialization. In particular, the rewards collected by the optimizer are essentially bounded by Lemma 13.

Lemma 15 *Assume that* $\Delta \geqslant \frac{1}{\eta}\log(2n^2T)$. *The cumulative reward collected by the optimizer, when playing against MWU is upper bounded by,*

$$2T^{1-\beta\varepsilon/3} + (k^\star(1+5\varepsilon)+9\Delta)\sum_{i=1}^{n} W(z_i, z_{i+1}) + (2k^\star + 2\Delta + 4) \tag{27}$$

for some Hamiltonian cycle $z_1 \to z_2 \to \cdots \to z_n \to z_1$ *on* G.

The same argument will apply for lower bounding the total reward collected by the best optimizer strategy. We will consider the optimizer which plays ♦, $(1-\varepsilon)T$ times first, prior to following the optimizer strategy outlined in Lemma 14. Note that the remaining duration in the epoch is εT which is at least $nk^\star$.

Lemma 16 *Consider any maximum weight Hamiltonian cycle in* G, *defined by the sequence of vertices* $z_1^\star \to z_2^\star \to \cdots \to z_n^\star \to z_1^\star$. *For any absolute constant* $\gamma \in (0,1)$, *there exists an optimizer strategy such that the total reward collected is at least,*

$$(1-\varepsilon)^3 k^\star \times \frac{1}{1+p^{-\varepsilon}}\left(1-\frac{2}{n}\right)\sum_{i=1}^{n} W(z_i^\star, z_{i+1}^\star). \tag{28}$$

We may now combine Lemmas 15 and 16 to prove Theorem 1, using the fact that both equations, up to an $\approx k^\star$ factor, capture the weight of a Hamiltonian cycle on G.

5. Conclusion and future questions

We show that it is hard in general to optimize against a no-regret learner: specifically, against Hedge/MWU. This implies that positive results can only be obtained for games with specific structure, as was extensively studied in prior literature. It remains open what is the best runtime for the optimizer. While it is not polynomial in the game size unless $\mathsf{P} = \mathsf{NP}$, can it be polynomial in T? Can the runtime depend exponentially only on $\min(|\mathcal{A}|, |\mathcal{B}|)$? Concretely, is there an algorithm that runs in time $\text{poly}(|\mathcal{A}|, |\mathcal{B}|, T)e^{O(\min(|\mathcal{A}|,|\mathcal{B}|))}$? Further, beyond the special cases studied, is there a broader class of games in which it is possible to efficiently find optimize the optimizer's reward? Beyond MWU, there is an efficient optimization algorithm against no-swap regret learners (Deng et al., 2019b), but what about other learners? Taking the learner's perspective, can one analyze how well it performs against optimizers and which learning algorithms are desired in such a scenario? See related work for some studies in this direction.

References

Ioannis Anagnostides, Constantinos Daskalakis, Gabriele Farina, Maxwell Fishelson, Noah Golowich, and Tuomas Sandholm. Near-optimal no-regret learning for correlated equilibria in multi-player general-sum games. In *ACM Symposium on Theory of Computing*, 2022a.

Ioannis Anagnostides, Ioannis Panageas, Gabriele Farina, and Tuomas Sandholm. On last-iterate convergence beyond zero-sum games. In *International Conference on Machine Learning*, 2022b.

Nivasini Ananthakrishnan, Nika Haghtalab, Chara Podimata, and Kunhe Yang. Is knowledge power? on the (im) possibility of learning from strategic interaction. *arXiv preprint arXiv:2408.08272*, 2024.

Sanjeev Arora, Elad Hazan, and Satyen Kale. The multiplicative weights update method: a meta-algorithm and applications. *Theory of computing*, 8(1):121–164, 2012.

Eshwar Ram Arunachaleswaran, Natalie Collina, and Jon Schneider. Pareto-optimal algorithms for learning in games. *arXiv preprint arXiv:2402.09549*, 2024.

Angelos Assos, Yuval Dagan, and Constantinos Costis Daskalakis. Maximizing utility in multi-agent environments by anticipating the behavior of other learners. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL https://openreview.net/forum?id=0uGlKYS7a2.

Maria-Florina Balcan, Avrim Blum, Nika Haghtalab, and Ariel D. Procaccia. Commitment without regrets: Online learning in stackelberg security games. In *Proceedings of the Sixteenth ACM Conference on Economics and Computation*, pages 61–78, 2015.

Mark Braverman, Jieming Mao, Jon Schneider, and Matt Weinberg. Selling to a no-regret buyer. In *Proceedings of the 2018 ACM Conference on Economics and Computation*, pages 523–538, 2018.

William Brown, Jon Schneider, and Kiran Vodrahalli. Is learning in games good for the learners? *Advances in Neural Information Processing Systems*, 36, 2024.

Linda Cai, S Matthew Weinberg, Evan Wildenhain, and Shirley Zhang. Selling to multiple no-regret buyers. In *International Conference on Web and Internet Economics*, pages 113–129. Springer, 2023.

Emilio Calvano, Giacomo Calzolari, Vincenzo Denicolo, and Sergio Pastorello. Artificial intelligence, algorithmic pricing, and collusion. *American Economic Review*, 110(10):3267–3297, 2020.

Nicolo Cesa-Bianchi and Gábor Lugosi. *Prediction, learning, and games*. Cambridge university press, 2006.

Yiling Chen and Tao Lin. Persuading a behavioral agent: Approximately best responding and learning. *arXiv preprint arXiv:2302.03719*, 2023.

Natalie Collina, Eshwar Ram Arunachaleswaran, and Michael Kearns. Efficient stackelberg strategies for finitely repeated games. In *Proceedings of the 2023 International Conference on Autonomous Agents and Multiagent Systems*, pages 643–651, 2023.

Vincent Conitzer and Tuomas Sandholm. Computing the optimal strategy to commit to. In *Proceedings of the 7th ACM Conference on Electronic Commerce*, pages 82–90, 2006.

Constantinos Daskalakis, Maxwell Fishelson, and Noah Golowich. Near-optimal no-regret learning in general games. *Advances in Neural Information Processing Systems*, 34:27604–27616, 2021.

Yuan Deng, Jon Schneider, and Balasubramanian Sivan. Prior-free dynamic auctions with low regret buyers. *Advances in Neural Information Processing Systems*, 32, 2019a.

Yuan Deng, Jon Schneider, and Balasubramanian Sivan. Strategizing against no-regret learners. *Advances in neural information processing systems*, 32, 2019b.

Heng Du and Tiaojun Xiao. Pricing strategies for competing adaptive retailers facing complex consumer behavior: Agent-based model. *International Journal of Information Technology & Decision Making*, 18(06):1909–1939, 2019.

Gabriele Farina, Ioannis Anagnostides, Haipeng Luo, Chung-Wei Lee, Christian Kroer, and Tuomas Sandholm. Near-optimal no-regret learning dynamics for general convex games. In *Neural Information Processing Systems (NeurIPS)*, 2022.

S. Freund and L. Gittins. Strategic learning and teaching in games. *Econometrica*, 66(3):597–625, 1998.

Yoav Freund and Robert E Schapire. A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of computer and system sciences*, 55(1):119–139, 1997.

Xinxiang Guo and Yifen Mu. The optimal strategy against hedge algorithm in repeated games. *arXiv preprint arXiv:2312.09472*, 2023.

Guru Guruganesh, Yoav Kolumbus, Jon Schneider, Inbal Talgam-Cohen, Emmanouil-Vasileios Vlatakis-Gkaragkounis, Joshua R Wang, and S Matthew Weinberg. Contracting with a learning agent. *arXiv preprint arXiv:2401.16198*, 2024.

Nika Haghtalab, Thodoris Lykouris, Sloan Nietert, and Alexander Wei. Learning in stackelberg games with non-myopic agents. In *Proceedings of the 23rd ACM Conference on Economics and Computation*, pages 917–918, 2022.

Marek Karpinski and Richard Schmied. On approximation lower bounds for tsp with bounded metrics. *arXiv preprint arXiv:1201.5821*, 2012.

Rachitesh Kumar, Jon Schneider, and Balasubramanian Sivan. Strategically-robust learning algorithms for bidding in first-price auctions. *arXiv preprint arXiv:2402.07363*, 2024.

Niklas Lauffer, Mahsa Ghasemi, Abolfazl Hashemi, Yagiz Savas, and Ufuk Topcu. No-regret learning in dynamic stackelberg games. In *arXiv preprint arXiv:2203.17184*, 2022.

Tao Lin and Yiling Chen. Persuading a learning agent. *arXiv preprint arXiv:2402.09721*, 2024.

Nick Littlestone and Manfred K Warmuth. The weighted majority algorithm. *Information and computation*, 108(2):212–261, 1994.

Yishay Mansour, Mehryar Mohri, Jon Schneider, and Balasubramanian Sivan. Strategizing against learners in bayesian games. In *Conference on Learning Theory*, pages 5221–5252. PMLR, 2022.

Janusz Marecki, Gerry Tesauro, and Richard Segal. Playing repeated stackelberg games with unknown opponents. In *Proceedings of the 11th International Conference on Autonomous Agents and Multiagent Systems - Volume 2*, pages 821–828, 2012.

Alireza Masoumian and James R Wright. Model selection for average reward rl with application to utility maximization in repeated games. *arXiv preprint arXiv:2411.06069*, 2024.

Denis Nekipelov, Vasilis Syrgkanis, and Eva Tardos. Econometrics for learning agents. In *Proceedings of the sixteenth acm conference on economics and computation*, pages 1–18, 2015.

Gali Noti and Vasilis Syrgkanis. Bid prediction in repeated auctions with learning. In *Proceedings of the Web Conference 2021*, pages 3953–3964, 2021.

Binghui Peng, Weiran Shen, Pingzhong Tang, and Song Zuo. Learning optimal strategies to commit to. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 2149–2156, 2019.

Georgios Piliouras, Ryann Sim, and Stratis Skoulakis. Beyond time-average convergence: Near-optimal uncoupled online learning via clairvoyant multiplicative weights update. *Advances in Neural Information Processing Systems*, 35:22258–22269, 2022.

Julia Robinson. An iterative method of solving a game. *Annals of mathematics*, pages 296–301, 1951.

Aviad Rubinstein and Junyao Zhao. Strategizing against no-regret learners in first-price auctions. In *Proceedings of the 25th ACM Conference on Economics and Computation*, pages 894–921, 2024.

Vasilis Syrgkanis, Alekh Agarwal, Haipeng Luo, and Robert E. Schapire. Fast convergence of regularized learning in games. In *Advances in Neural Information Processing Systems*, volume 28, 2015.

J. Youn. Learning algorithms in games. *Journal of Game Theory*, 56(2):123–134, 2004.

Brian Hu Zhang, Gabriele Farina, Ioannis Anagnostides, Federico Cacciamani, Stephen Marcus McAleer, Andreas Alexander Haupt, Andrea Celli, Nicola Gatti, Vincent Conitzer, and Tuomas Sandholm. Computing optimal equilibria and mechanisms via learning in zero-sum extensive-form games. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023a.

Brian Hu Zhang, Gabriele Farina, Ioannis Anagnostides, Federico Cacciamani, Stephen Marcus McAleer, Andreas Alexander Haupt, Andrea Celli, Nicola Gatti, Vincent Conitzer, and Tuomas Sandholm. Steering no-regret learners to optimal equilibria. 2023b.